

The Endowment Guide

How to give a dividend-paying coin a permanent holder that never sells, funded by its own largest holders.

Source: <https://github.com/The-PENIS-Endowment> (Apache-2.0)

1. The idea in brief

Most meme coins pay holders nothing, so a large holder only gets paid by selling. Coins that pay dividends change that. A large position becomes an income-producing asset, and holding becomes the profitable move. We call holders who accumulate for the income **landlords**: traders create volume, volume pays rent, and rent keeps supply off the market.

An **endowment** takes the idea one step further. Landlords lend it their dividend income, and it turns that income into more of the coin, held in a vault that can never sell. It is a landlord whose commitment is written into code.

2. How an endowment works

1. **Delegate.** A landlord signs one transaction that delegates their dividend token account (for example their PUMP account) to the endowment. Only that one account is involved. Their coin, their SOL and every other token stay untouched.
 2. **Sweep.** When dividends land in the landlord's wallet, anyone can trigger a sweep. It moves the new dividend tokens, above the balance the landlord held when they joined, into the endowment's dividend vault.
 3. **Buy.** The endowment buys the coin in small amounts through the coin's Raydium CPMM pool and locks it in its coin vault. No instruction can ever move coin out of that vault.
 4. **Leave.** A landlord can revoke the delegation at any time, from any wallet, without the endowment's permission.
-

3. The rules

Commitment threshold. Sweeps run only while landlords together hold enough of the coin's supply, 30% by default. Once a day, anyone can run the commitment count. It starts a round, reads every landlord in batches, and closes the round once all are read (or after two hours). Each landlord counts for

the lower of their balance now and at their previous reading, so coin must be held across a full day to count, and only landlords whose delegation is still in place are counted. Between counts, anyone can refresh landlords' recorded balances, which can only lower them, so coin moved between landlord wallets counts once. Every landlord's counted amount is published on-chain after each count. Sweeps switch on at the threshold and pause if commitment falls below a lower line (25% by default), so small changes don't flip them back and forth.

Landlords. There is no limit on landlords. Each must hold a minimum stake (0.1% of supply by default); anyone can remove a landlord who no longer holds it or has revoked.

Commitment is per wallet. Everything in a landlord's wallet is committed: all its coin counts toward the threshold, and all new dividend tokens arriving in it are swept. To commit part of a holding, keep the rest in another wallet.

Buying. Each buy is sized by the contract: no larger than the per-buy cap, the daily allowance, or what the pool can absorb within the price-impact limit. Buys are priced against the pool's 10-minute time-weighted average price, and refused if the spot price has been pushed away from it. Buys are spaced by a minimum interval, and a small tip (0.25% by default) pays whoever triggers each buy, which covers the automation's network fees.

Milestone. Each endowment has a milestone, for \$PENIS 200 million coins bought. After it, contributions keep flowing, and a set share of every buy becomes permanent liquidity in the pool. The liquidity tokens are held by the endowment and can never be withdrawn.

Pause. A guardian key can pause the endowment for up to seven days. A pause can't move funds, lifts on its own, and can't be renewed back to back. Landlords can always leave while paused.

Timelock. Every limit and threshold is bounded by hard-coded limits, and any change waits 72 hours on-chain before it takes effect.

Key burn. After a public testing period the admin role is renounced and the program's upgrade key is destroyed. From then on, no one can change the rules or the code.

4. Creating an endowment on the shared contract

The shared contract hosts many endowments. Each one has its own vaults, landlords, settings and count, keyed by its coin and its creator, and no endowment can touch another's accounts.

Requirements

- A Raydium CPMM pool pairing your coin with the asset it pays as dividends.

- Dividends pushed to holders' wallets automatically (no claim step). stonk.fun reward coins work this way.
- Either token standard (SPL Token or Token-2022) for the coin and the dividend asset.

Steps

1. Choose your settings: buy caps, daily allowance, price-impact limit, minimum buy, buy interval, tip, activation threshold, minimum stake, milestone, and the post-milestone liquidity share.
2. Choose an optional donation to the \$PENIS Endowment: 0%, 0.1%, 0.2% or 0.3% of each buy. It is available for coins paid in PUMP and is locked at creation.
3. Send `create_endowment` from your creator wallet. Put the admin and guardian roles in multisigs you control, such as Squads vaults.
4. Share your endowment's opt-in link with your landlords, and run a keeper (section 6), or rely on anyone who wants the tip.

Creation opens on the shared contract once its upgrade key has been destroyed, so every project runs on code that can never change.

5. Deploying your own copy

Everything is Apache-2.0, so you can run your own deployment:

1. Clone the `endowment` repository. Install Rust, the Solana CLI and Anchor 1.1.2.
2. `anchor build`, then `cargo test` to run the unit and integration tests. The integration tests use snapshots of a real Raydium pool.
3. Update the program ID, and the flagship constants if you want donations to go elsewhere, then deploy with a fresh upgrade authority held in a multisig.
4. Clone the `website` repository for the opt-in page and keeper, and point it at your program ID and instance.
5. Run your own review before launch, test with small caps, then renounce the admin role and destroy the upgrade key.

6. Running the keeper

The keeper is a small service that triggers the endowment's permissionless instructions. It holds no special power, and anyone can run one.

- **Sweeps.** Run after every dividend drop (for example from a webhook on the dividend distributor), with a timer as a fallback. Each landlord is swept on its own, so one failure never blocks the

others.

- **Buys.** Check every few minutes. The keeper waits until the vault holds at least the minimum buy and a randomized time after the last buy has passed, then calls `buyback`. The contract does the sizing and pricing.
- **Count.** Once a day, run `begin_count`, then `count_landlords` in batches of about eight landlords, then `finish_count`. A batch fits a normal transaction, so no lookup table is needed.
- **Refresh.** A few times a day, at random times, run `refresh_landlords` over every landlord in randomly grouped batches.

The website repository includes a ready-made keeper with setup notes in `docs/keeper.md`.

7. Security notes

- **What landlords approve.** Landlords approve an unlimited delegation on one dividend account. The program only ever moves tokens above the landlord's recorded balance, and only into its own vault. Before the key burn, the upgrade key holder could change the program, so keep that key in a multisig and announce every upgrade.
- **Recommend a dedicated wallet.** Once a landlord joins, all new dividend tokens arriving in that account go to the endowment, whatever their source. A wallet that holds only the coin keeps this simple.
- **Check your coin and dividend asset for mint controls.** Transfer-fee authorities, transfer hooks, pause or freeze authorities, and permanent delegates all matter. The contract fails closed on excessive fees, an enabled dividend transfer hook, and a disabled pool, but it can't prevent a third party from using those controls.
- **Test with small caps first,** and review the code yourself or with your own agents before you launch.